

Interactive Real-time Leader-Follower UAV/UGV 复现实验流程

本文档面向当前目录 /home/robbinpanda/GKCII，目标是复现论文 **Interactive Real-time Leader Follower Control System for UAV and UGV** 的主要算法流程，而不是 1:1 复现 QUANSER QDrone/QBot、OptiTrack、MATLAB/Simulink/QUARC 硬件系统。

推荐复现路线是：用 Ubuntu 20.04 + ROS Noetic + Gazebo Classic + PX4 SITL，在 catkin_ws 和 damad_ws 的现有 ROS 代码上完成“UAV leader 航点飞行 + UGV follower 跟随 + 障碍物网格记录与绕障”的仿真实验。

1. 当前文件检查结论

1.1 可以直接利用的内容

路径	用途	复现价值
Interactive_Real-time_Leader_Follower_Control_System_for_UAV_and_UGV.pdf	目标论文	已完整下载，8 页，可作为算法依据
catkin_ws/src/site_model	Gazebo 场景、车辆模型、UAV/UGV 控制脚本、传感器融合脚本	主复现工作空间
catkin_ws/uav2.launch	两架 PX4 Iris UAV 的 MAVROS/SITL 启动文件	可改成单 UAV 或保留双 UAV
catkin_ws/2cars.sh	两车 + UAV 仿真启动脚本	可作为综合启动模板
catkin_ws/uav_cruise.sh	双 UAV 航点巡航启动脚本	可复用 UAV 航点控制逻辑
catkin_ws/src/site_model/src/iris0_cruise.py	UAV0 起飞、定高、按航点飞行	可改造成论文中的 leader
catkin_ws/src/site_model/src/platoon.py	后车跟随前车里程计轨迹	可改造成 UGV 跟随 UAV 投影坐标
catkin_ws/models/{ground_plane,cantilevered_light,gazebo_traffic_light}	脚本明确要求复制到 Gazebo models	有用，应保留
.gazebo/models/tagcode	二维码/AprilTag 相关模型	damad_ws 二维码跟随实验可能需要
damad_ws/src/control_pkg	多 UAV/UGV、二维码跟随、odom 跟随、键盘/G29 控制	可作为备选参考

1.2 明显缺失或需要重新获得的内容

缺失项	影响	处理建议
~/PX4-Autopilot 不在当前项目内	所有 roslaunch px4 ... 都无法运行	必须重新下载 PX4-Autopilot 并编译
PX4-Autopilot/launch/ uav0.launch、uav1.launch、ugv0.launch、ugv1.launch	damad_ws/demo0.sh 到 demo4.sh 会调用这些文件	如果云端有这些 launch 文件，应补下载；否则按 catkin_ws/uav2.launch 和 PX4 多机模板重建
PX4-Autopilot/Tools/simulation/gazebo-classic/ sitr_gazebo-classic/worlds/bl-ver6.world 的本地修改	damad_ws/README.md 提到需要修改红绿灯位置/数量	如果要跑 damad_ws 原始 demo，应从原云端补齐或手工重做
论文原始 Simulink/QUARC 模型	无法 1:1 复现硬件实验	本实验用 ROS 节点代替 Mission Server、QDrone Commander、QBot Commander
QBot HC-SR04 超声传感器代码	原文绕障输入缺失	Gazebo 中建议改用双 ray 传感器模拟超声，或先用障碍物真值调试算法

1.3 可以不下载或不保留的内容

内容	判断
catkin_ws/build、catkin_ws/devel、damad_ws/build、damad_ws/devel 大量 *:Zone.Identifier 文件 .gazebo/client-*.gazebo/server-*.gazebo/ogre.log .gazebo/models/model.tar.gz.* .opencode/node_modules .gazebo/models 中大部分通用模型	云端拷贝来的构建产物不可依赖，换机器后应重新 catkin_make Windows 下载附带的标记文件，Linux/ROS 不需要 Gazebo 日志，不是必要资源 模型下载缓存包，不是运行必需 与 ROS/Gazebo 复现实验无关 不是全部有用。保留被 world 文件引用的模型即可，例如 ground_plane、sun、tagcode、cantilevered_light、gazebo_traffic_light、障碍物模型等

2. 论文主要算法拆解

论文系统由 3 个逻辑模块组成：

- 1. **Leader UAV**：起飞到至少 0.5 m，高度稳定后按预设航点移动。每到一个航点附近后，等待 follower 靠近，再飞向下一个航点。最后返回工作区中心降落。
- 2. **Mission Server**：读取 leader 和 follower 位姿，负责航点切换和安全检查。如果 OptiTrack 失效或急停触发，停止模型。
- 3. **Follower UGV**：读取 leader/follower 位姿，生成跟随轨迹。没有障碍物时直接朝 leader 的地面投影移动；检测到障碍物时，在二维栅格地图中记录障碍并重新规划绕障路径。

原文绕障参数：

参数	原文设置	仿真建议
工作区	5 m x 3 m	可用 6 m x 4 m 或按 Gazebo 场景缩放
栅格地图	60 x 40	每格 0.1 m
到 leader 停止/切换距离	0.25 m	先用 0.25 m，仿真不稳定可放宽到 0.4 m
障碍触发距离	0.30 m	ray 传感器检测小于 0.30 m
UGV 半径膨胀	0.175 m	栅格中膨胀 2 格
双传感器障碍宽度	约 0.6 m	左/右传感器分别标记 3 格

3. 环境配置方法

3.1 推荐系统

使用 **Ubuntu 20.04 LTS**。不要优先使用 Ubuntu 22.04/24.04，因为当前代码明显依赖 ROS Noetic、Gazebo Classic、MAVROS 和 PX4 Gazebo Classic。PX4 官方文档说明 Gazebo Classic 对 PX4 的支持主要到 Ubuntu 20.04，Ubuntu 22.04 及以后应使用新版 Gazebo；这会和当前代码中的 gazebo-classic 路径、launch 文件不匹配。

参考：

- PX4 Gazebo Classic 文档：https://docs.px4.io/v1.15/en/sim_gazebo_classic/
- PX4 Ubuntu/ROS Noetic 配置文档：https://docs.px4.io/v1.14/en/dev_setup/dev_env_linux_ubuntu
- MAVROS GeographicLib 数据说明：<https://docs.ros.org/en/rolling/p/mavros/>

3.2 安装基础工具

```
sudo apt update
sudo apt install -y \
  git wget curl lsb-release gnupg2 software-properties-common \
  build-essential cmake python3-pip python3-rosdep python3-catkin-tools \
  python3-vcstool python3-osrf-pycommon gnome-terminal
```

3.3 安装 ROS Noetic

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'
curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key add -
sudo apt update
sudo apt install -y ros-noetic-desktop-full
```

```
sudo rosdep init
rosdep update
```

```
echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

如果 sudo rosdep init 提示已经初始化，跳过该步，直接 rosdep update。

3.4 安装本项目常用 ROS 依赖

```
sudo apt install -y \
  ros-noetic-gazebo-ros \
  ros-noetic-gazebo-ros-control \
  ros-noetic-gazebo-plugins \
  ros-noetic-mavros \
  ros-noetic-mavros-extras \
  ros-noetic-ackermann-msgs \
  ros-noetic-controller-manager \
  ros-noetic-joint-state-controller \
  ros-noetic-effort-controllers \
  ros-noetic-velocity-controllers \
  ros-noetic-position-controllers \
  ros-noetic-cv-bridge \
  ros-noetic-image-transport \
  ros-noetic-pcl-ros \
  ros-noetic-pcl-conversions \
  ros-noetic-pcl-msgs \
  ros-noetic-tf \
  ros-noetic-tf2-ros \
  ros-noetic-tf2-geometry-msgs \
  ros-noetic-robot-state-publisher \
```

```
ros-noetic-joint-state-publisher \  
ros-noetic-joint-state-publisher-gui \  
ros-noetic-xacro \  
ros-noetic-rviz \  
libpcl-dev libopencv-dev protobuf-compiler libeigen3-dev
```

安装 MAVROS GeographicLib 数据:

```
sudo /opt/ros/noetic/lib/mavros/install_geographiclib_datasets.sh
```

3.5 安装 PX4-Autopilot

建议先使用和 Gazebo Classic/ROS1 更匹配的 PX4 1.14 或 1.15 分支。以下示例用 v1.14.3, 如果你的课程或实验给了指定版本, 以指定版本为准。

```
cd ~  
git clone https://github.com/PX4/PX4-Autopilot.git --recursive  
cd ~/PX4-Autopilot  
git checkout v1.14.3  
git submodule update --init --recursive
```

```
bash ./Tools/setup/ubuntu.sh --no-sim-tools --no-nuttx  
sudo apt install -y gazebo libgazebo11 libgazebo-dev
```

重启后验证:

```
cd ~/PX4-Autopilot  
make px4_sitl gazebo-classic
```

如果能打开 Gazebo 并生成 Iris, 则 PX4/Gazebo Classic 基础环境可用。

3.6 配置 PX4 与本项目的 launch/model 路径

当前 catkin_ws/2cars.sh 和 catkin_ws/uav_cruise.sh 注释要求把 uav2.launch 复制到 PX4 的 launch 目录:

```
cp ~/GKCII/catkin_ws/uav2.launch ~/PX4-Autopilot/launch/uav2.launch
```

当前 damad_ws/demo*.sh 还会调用:

```
roslaunch px4 ugv0.launch  
roslaunch px4 ugv1.launch  
roslaunch px4 uav0.launch  
roslaunch px4 uav1.launch
```

这些文件当前没有下载到 /home/robbinpanda/GKCII。如果云端原目录中有它们, 需要补下载并复制到:

~/PX4-Autopilot/launch/

如果云端没有, 就基于 catkin_ws/uav2.launch 和 PX4 single_vehicle_spawn.launch 重建。最低限度可以先不跑 damad_ws/demo*.sh, 优先跑通 catkin_ws。

配置 Gazebo 模型:

```
mkdir -p ~/.gazebo/models
cp -r ~/GKCII/catkin_ws/models/* ~/.gazebo/models/
```

如果使用当前下载的 .gazebo/models:

```
cp -r ~/GKCII/.gazebo/models/tagcode ~/.gazebo/models/ 2>/dev/null || true
cp -r ~/GKCII/.gazebo/models/gazebo_traffic_light ~/.gazebo/models/ 2>/dev/null || true
cp -r ~/GKCII/.gazebo/models/cantilevered_light ~/.gazebo/models/ 2>/dev/null || true
```

3.7 编译两个工作空间

不要依赖云端下载的 build/ 和 devel/。先重新安装依赖，再编译。

```
cd ~/GKCII/catkin_ws
source /opt/ros/noetic/setup.bash
rosdep install --from-paths src --ignore-src -r -y
catkin_make
source devel/setup.bash

cd ~/GKCII/damad_ws
source /opt/ros/noetic/setup.bash
rosdep install --from-paths src --ignore-src -r -y
catkin_make
source devel/setup.bash
```

如果 catkin_ws 因 OpenPCDet/CUDA 相关内容报错，先只编译复现实验需要的包：

```
cd ~/GKCII/catkin_ws
catkin_make -DCATKIN_WHITELIST_PACKAGES="msgs;per_msgs;pkg;site_model;radar_plugin"
```

如果提示找不到 radar_plugin，先去掉它：

```
catkin_make -DCATKIN_WHITELIST_PACKAGES="msgs;per_msgs;pkg;site_model"
```

4. 跑通现有 .sh 脚本的方法

4.1 运行前通用检查

所有脚本默认使用 gnome-terminal 开多个窗口，因此需要桌面环境。如果在 SSH/headless 环境运行，应改成手动开多个终端或改写为 tmux。

```
cd ~/GKCII/catkin_ws
chmod +x *.sh
source /opt/ros/noetic/setup.bash
source devel/setup.bash
```

单独检查 ROS 包是否可见：

```
rospack find site_model
rospack find pkg
rospack find mavros
rospack find px4
```

```
rospack find px4 找不到时, 说明 PX4 的 ROS 路径没有配置。临时配置:
export ROS_PACKAGE_PATH=$ROS_PACKAGE_PATH:~/PX4-Autopilot
export ROS_PACKAGE_PATH=$ROS_PACKAGE_PATH:~/PX4-Autopilot/Tools/simulation/gazebo-classic/sitl_gazebo-classic

建议写入 ~/.bashrc:

echo 'export ROS_PACKAGE_PATH=$ROS_PACKAGE_PATH:~/PX4-Autopilot' >> ~/.bashrc
echo 'export ROS_PACKAGE_PATH=$ROS_PACKAGE_PATH:~/PX4-Autopilot/Tools/simulation/gazebo-classic/sitl_gazebo-classic' >> ~/.bashrc
```

4.2 catkin_ws 脚本

脚本	作用	前置条件	复现相关度
uav_cruise.sh	启动 Gazebo 场景、PX4 双 UAV、两个 UAV 航点巡航脚本	uav2.launch 已复制到 PX4; PX4/MAVROS 可用	高, 可验证 leader 航点飞行
2cars.sh	启动两车场景、双 UAV、UAV 跟随小车、交通灯、两车 platoon、巡线	uav2.launch、Gazebo 模型、site_model 编译成功	高, 可作为综合启动模板
2cars_1.sh	两车、交通灯、舵机控制、后车跟随、雷达/相机融合	传感器融合依赖较多	中
tag_detect.sh	两车场景 + AprilTag 检测	tagcode 模型、相机话题可用	低到中
yolo_detect.sh	两车场景 + YOLO 检测	ultralytics、yolo11n.pt	低
lidar_cam.sh	激光雷达点云和相机相关节点	PCL、OpenPCDet 可选	低
radar_cam.sh	毫米波雷达 + 相机融合	radar_plugin 编译成功	低

推荐先跑最小闭环:

```
cd ~/GKCII/catkin_ws
source devel/setup.bash
./uav_cruise.sh
```

观察:

```
rostopic list | grep mavros
rostopic echo /uav0/mavros/local_position/pose
rostopic echo /uav1/mavros/local_position/pose
```

再跑两车模板:

```
cd ~/GKCII/catkin_ws
source devel/setup.bash
./2cars.sh
```

观察:

```
rostopic echo /car1/base_pose_ground_truth
rostopic echo /car2/base_pose_ground_truth
rostopic echo /car2/ackermann_cmd_mux/output
```

如果脚本卡住或某个窗口报错，按脚本里的命令拆开手动跑。例如 2cars.sh 可以拆成：

```
cd ~/GKCII/catkin_ws
source devel/setup.bash
roslaunch site_model spawn_2cars.launch
```

另开终端：

```
roslaunch px4 uav2.launch
```

另开终端：

```
cd ~/GKCII/catkin_ws
source devel/setup.bash
roslaunch site_model iris0_follow.py
```

4.3 damad_ws 脚本

脚本	作用	当前阻塞点
demo0.sh	1 UGV + 2 UAV，UAV 识别二维码跟随 UGV	缺 px4/ugv0.launch、px4/uav0.launch
demo1.sh	2 UGV + 2 UAV，二维码跟随	缺 px4/ugv1.launch、px4/uav1.launch
demo2.sh	1 UGV + 2 UAV，UAV 订阅 odom 跟随	缺 PX4 launch
demo3.sh	2 UGV + 2 UAV，odom 跟随和后车跟随	缺 PX4 launch
demo4.sh	键盘控制 UAV/UGV	缺 PX4 launch
g29_info.sh	罗技 G29 信息读取	需要硬件设备

在补齐 PX4 launch 前，不建议优先用 damad_ws 做论文复现。它的代码方向多为 **UAV 跟随 UGV**，而论文是 **UGV 跟随 UAV**。可保留作为键盘控制、二维码识别、odom 跟随代码参考。

5. 复现论文主要算法的步骤

5.1 第一阶段：跑通基础仿真

目标：证明 Gazebo、ROS、PX4、MAVROS、UGV Ackermann 控制链路都能工作。

- 1. 跑通 PX4 单机：

```
cd ~/PX4-Autopilot
make px4_sitl gazebo-classic
```

- 2. 跑通 catkin_ws 两车场景：

```
cd ~/GKCII/catkin_ws
source devel/setup.bash
roslaunch site_model spawn_2cars.launch
```

- 3. 跑通双 UAV：

```
roslaunch px4 uav2.launch
```

4. 跑通现有 UAV 巡航：

```
cd ~/GKCII/catkin_ws
source devel/setup.bash
roslaunch site_model iris0_cruise.py
```

5. 跑通现有两车跟随：

```
cd ~/GKCII/catkin_ws
source devel/setup.bash
roslaunch site_model platoon.py car1 car2 base_link2 follow1
```

阶段验收：

- Gazebo 中至少有 1 架 Iris UAV 和 1 辆 UGV。
- /uav0/mavros/local_position/pose 有连续输出。
- /car1/base_pose_ground_truth 或 /car2/base_pose_ground_truth 有连续输出。
- 发布到 /carX/ackermann_cmd_mux/output 后 UGV 能运动。

5.2 第二阶段：实现论文的 leader UAV

复用 catkin_ws/src/site_model/src/iris0_cruise.py。

建议新建节点：

```
catkin_ws/src/site_model/src/leader_uav_waypoint.py
```

核心逻辑：

1. 订阅 /uav0/mavros/local_position/pose。
2. 调用 /uav0/mavros/set_mode，进入 OFFBOARD。
3. 调用 /uav0/mavros/cmd/arming 解锁。
4. 起飞到目标高度，例如 1.5 m。
5. 从 Mission Server 接收当前航点，或读取参数 ~waypoints。
6. 用比例控制生成速度：

```
vx = kp * (target_x - current_x)
vy = kp * (target_y - current_y)
vz = kp_z * (target_z - current_z)
```

7. 发布到：

```
/uav0/mavros/setpoint_velocity/cmd_vel_unstamped
```

8. 最后一个航点完成后执行 AUTO.LAND。

注意：现有 iris0_cruise.py 是内部自己切换航点。为更接近论文，应把“是否切换下一航点”的判断交给 Mission Server：只有 follower 距 leader 小于 0.25 m 时才切换。

5.3 第三阶段：实现 Mission Server

建议新建：

catkin_ws/src/site_model/src/mission_server.py

订阅：

/uav0/mavros/local_position/pose
/car1/base_pose_ground_truth

发布：

/leader/current_waypoint	geometry_msgs/PoseStamped
/mission/state	std_msgs/String
/mission/start	std_msgs/Bool

核心逻辑：

```
waypoints = [  
    (0.0, -1.0, 1.5),  
    (1.5, -1.0, 1.5),  
    (1.5, 1.0, 1.5),  
    (0.0, 0.0, 1.5),  
]  
  
if leader_takeoff_ok and mission_started:  
    publish current waypoint  
  
if distance_xy(ugv, uav) < 0.25:  
    switch to next waypoint  
  
if last waypoint reached and follower close:  
    publish home/land command
```

这里的 UGV 位姿用 Gazebo ground truth 代替论文中的 OptiTrack。算法层面等价于“外部定位系统提供 leader/follower 位姿”。

5.4 第四阶段：实现 UGV follower

复用 platoon.py 的姿态计算和 Ackermann 输出方式，但把目标从“前车轨迹队列”改成“UAV 当前地面投影或 Mission Server 给出的局部路径点”。

建议新建：

catkin_ws/src/site_model/src/ugv_follow_uav.py

订阅：

/uav0/mavros/local_position/pose
/car1/base_pose_ground_truth
/ugv/local_goal

发布：

```
/car1/ackermann_cmd_mux/output
```

无障碍时控制律：

```
dx = target_x - ugv_x
dy = target_y - ugv_y
target_heading = atan2(dy, dx)
heading_error = wrap_to_pi(target_heading - ugv_yaw)
distance = sqrt(dx^2 + dy^2)

speed = clamp(kv * distance, 0.0, max_speed)
steering = clamp(kw * heading_error, -max_steer, max_steer)
```

```
if distance < 0.25:
    speed = 0
```

建议初始参数：

```
kv = 0.25
kw = 0.8
max_speed = 0.6 m/s
max_steer = 0.7 rad
```

5.5 第五阶段：实现障碍物检测输入

原文使用两个 HC-SR04 超声传感器。Gazebo 中建议用两个 ray 传感器模拟：

1. 在 pkg/urdf/car1.urdf 或对应 xacro 中给 UGV 前方加两个 ray/link：
 - front_left_ultrasonic
 - front_right_ultrasonic
2. 左右传感器都朝车头方向。
3. 最大距离设置 0.5 m 到 1.0 m。
4. 当距离小于 0.30 m 时认为检测到障碍。

如果暂时不想改 URDF，可以先用简化方案：

- 在 Gazebo world 中放固定 unit_box 障碍。
- 节点读取 /gazebo/model_states，用障碍物真值判断 UGV 前方 0.30 m 是否有障碍。
- 算法跑通后再换成 ray 传感器输入。

5.6 第六阶段：实现论文式网格绕障

建议新建：

```
catkin_ws/src/site_model/src/grid_obstacle_planner.py
```

内部数据：

```
map_width = 60
map_height = 40
```

```
resolution = 0.1
origin = (-3.0, -2.0)
grid[y][x] = 0 free, 1 occupied
```

世界坐标转栅格：

```
gx = int((world_x - origin_x) / resolution)
gy = int((world_y - origin_y) / resolution)
```

检测到障碍时：

1. 用 UGV 当前 yaw 计算前方 0.30 m 的中心点。
2. 如果左传感器触发，标记中心偏左约 0.30 m 的 3 格。
3. 如果右传感器触发，标记中心偏右约 0.30 m 的 3 格。
4. 对障碍做膨胀，半径约 2 格，对应 UGV 半径 0.175 m。

规划：

- 输入：UGV 当前栅格、UAV 地面投影栅格。
- 算法：A* 或 BFS。论文没有强调最优性，BFS 已足够；A* 更平滑。
- 输出：路径中的第 3 到第 6 个栅格作为局部目标，发布给 `ugv_follow_uav.py`。

当没有障碍时，局部目标直接设为 UAV 的地面投影。

5.7 第七阶段：组织最终复现实验

推荐最终实验启动顺序：

终端 1：

```
cd ~/GKCII/catkin_ws
source devel/setup.bash
roslaunch site_model spawn_2cars.launch
```

终端 2：

```
roslaunch px4 uav2.launch
```

终端 3：

```
cd ~/GKCII/catkin_ws
source devel/setup.bash
roslaunch site_model mission_server.py
```

终端 4：

```
cd ~/GKCII/catkin_ws
source devel/setup.bash
roslaunch site_model leader_uav_waypoint.py
```

终端 5：

```
cd ~/GKCII/catkin_ws
source devel/setup.bash
roslaunch site_model grid_obstacle_planner.py
```

终端 6:

```
cd ~/GKCII/catkin_ws
source devel/setup.bash
roslaunch site_model ugv_follow_uav.py
```

记录数据:

```
roslaunch record -0 interactive_realtime_repro.bag \
  /uav0/mavros/local_position/pose \
  /car1/base_pose_ground_truth \
  /car1/ackermann_cmd_mux/output \
  /mission/state \
  /ugv/local_goal
```

验收指标:

指标	目标
UAV 起飞与定高	起飞后保持 1.5 m 左右
UAV 航点	能按预设航点移动
Mission Server	follower 靠近 leader 后才切下一个航点
UGV 跟随	无障碍时能靠近 UAV 地面投影 0.25 m 到 0.4 m
绕障	遇到障碍后栅格地图出现占据区域，UGV 绕行后继续靠近 leader
安全	无定位数据或急停时 UGV 停车、UAV 不继续前进

6. 推荐的最小实现清单

如果只为了完成课程实验和论文主要算法复现，建议只新增 4 个脚本:

新脚本	作用	复用来源
mission_server.py	航点管理、leader/follower 距离判断	论文 Fig. 4
leader_uav_waypoint.py	UAV 起飞、定高、跟踪当前航点	iris0_cruise.py
ugv_follow_uav.py	UGV 跟随 UAV 地面投影或局部目标	platoon.py
grid_obstacle_planner.py	60x40 栅格、障碍记录、A*/BFS 绕障	论文 Fig. 6 和绕障段落

不建议一开始就引入 YOLO、OpenPCDet、雷达相机融合、二维码识别。这些不是论文主算法，会显著增加环境复杂度。

7. 常见问题处理

7.1 roslaunch px4 uav2.launch 找不到 px4

检查：

```
echo $ROS_PACKAGE_PATH
rospack find px4
```

修复：

```
export ROS_PACKAGE_PATH=$ROS_PACKAGE_PATH:~/PX4-Autopilot
export ROS_PACKAGE_PATH=$ROS_PACKAGE_PATH:~/PX4-Autopilot/Tools/simulation/gazebo-classic/sitl_gazebo-classic
```

7.2 Gazebo 报模型缺失

先复制项目模型：

```
mkdir -p ~/.gazebo/models
cp -r ~/GKCII/catkin_ws/models/* ~/.gazebo/models/
```

再按报错中的 model://xxx 从 ~/GKCII/.gazebo/models/xxx 复制。

7.3 catkin_make 找不到 per_msgs 或 msgs

确认它们在 catkin_ws/src：

```
ls ~/GKCII/catkin_ws/src/per_msgs
ls ~/GKCII/catkin_ws/src/msgs
```

然后清理 CMake 缓存重新编译：

```
cd ~/GKCII/catkin_ws
catkin_make clean
catkin_make
```

7.4 脚本打开多个窗口但没有继续执行

.sh 使用 gnome-terminal，每个窗口各自运行。看每个窗口的报错，不要只看原终端。若不方便，用手动拆命令方式运行。

7.5 UAV 无法起飞或 OFFBOARD 失败

MAVROS/PX4 常见要求是先持续发布 setpoint，再切 OFFBOARD。现有 iris0_cruise.py 在起飞流程里不断发布空 Twist，不要删除这类发布逻辑。检查：

```
rostopic echo /uav0/mavros/state
rostopic hz /uav0/mavros/setpoint_velocity/cmd_vel_unstamped
```

7.6 UGV 不动

检查控制器和话题：

```
rostopic list | grep ackermann  
rostopic echo /car1/ackermann_cmd_mux/output
```

如果发布了 Ackermann 但车不动，通常是 Gazebo 控制器未加载或命名空间不一致。优先对照 spawn_2cars.launch 中的 car1、car2 命名空间。

8. 最终建议

1. **先用 catkin_ws 复现主算法**，不要一开始跑 damad_ws。damad_ws 当前缺 PX4 launch 文件，而且主要逻辑方向和论文相反。
2. **必须补齐或安装 PX4-Autopilot**。当前仓库没有 PX4 主体，这是最大缺口。
3. **.gazebo 有部分有用，但不需要全部保留**。真正重要的是 world 文件引用到的模型；日志、缓存包和大量通用模型可不下载。
4. **不要依赖云端的 build/devel**。环境配置完成后重新编译。
5. **论文复现重点是算法闭环**：UAV 航点 leader、Mission Server 航点切换、UGV 跟随、障碍栅格记录与路径重规划。传感器融合、YOLO、雷达等不是本文主要贡献。